



Specifica Tecnica

Autore Alessandro Morabito, Alessandro Frison

Verificatore Alessandro Morabito

Approvazione

Registro delle versioni

Versione	Data	Autore	Verificatore	Descrizione delle modifiche
0.2.0	25/03/2026	Alessandro Frison	Alessandro Morabito	Aggiunta sezione introduzione e sezione Tecnologie
0.1.0	24/03/2026	Alessandro Morabito		Creazione documento

Indice

1	Introduzione	3
1.1	Scopo del documento	3
1.2	Glossario	3
1.3	Scopo del prodotto	3
1.4	Riferimenti	3
1.4.1	Riferimenti normativi	3
1.4.2	Riferimenti informativi	4
2	Tecnologie	4
2.1	Linguaggi di programmazione	4
2.1.1	TypeScript	4
2.2	Framework	4
2.2.1	NestJS	4
2.2.2	Langgraph	4
2.3	Librerie	4
2.3.1	ReactJS	4
2.3.2	TanstackRouter	5
2.3.3	mongoose	5
2.3.4	axios	5
2.3.5	isomorphic-git	5
2.3.6	react-markdown	5
2.3.7	dotenv	5
2.4	Testing	5
2.4.1	jest	5
2.4.2	vitest	6
2.4.3	playwright	6
2.5	Infrastruttura	6
2.5.1	Docker	6
2.5.2	AWS Amplify	6
2.5.3	AWS ECS	6
2.5.4	AWS EC2	6
3	Architettura	6
3.1	Architettura di deployment	6
3.2	Architettura logica	6
3.2.1	Frontend	7
3.2.2	Backend	7
3.2.3	Container	7

1 Introduzione

1.1 Scopo del documento

Il presente documento si pone l'obiettivo di definire in modo esaustivo la **Specifica Tecnica** del progetto, fornendo una guida dettagliata alle scelte architetture e implementative adottate. Esso rappresenta il ponte logico tra i requisiti iniziali e l'effettiva realizzazione del software, garantendo che ogni componente sia integrata in modo coerente e scalabile.

Il documento approfondisce l'evoluzione del sistema rispetto al **Proof of Concept (PoC)** iniziale, delineando una struttura robusta e orientata alla manutenibilità a lungo termine.

Nello specifico, questo documento si propone di:

- **Delineare l'architettura di sistema:** fornendo una panoramica delle macro-componenti e del flusso di dati tra di esse;
- **Dettagliare le scelte tecnologiche:** giustificando l'adozione di specifici framework, librerie e strumenti di sviluppo;
- **Illustrare i design pattern:** descrivendo gli schemi logici e le best practice utilizzate per risolvere problematiche ricorrenti e ottimizzare la struttura del codice;
- **Definire i processi di deployment:** spiegando come le diverse componenti vengono distribuite, configurate e rese operative nell'ambiente di produzione;
- **Fornire una base per la manutenzione:** offrendo la documentazione necessaria affinché futuri sviluppatori possano comprendere ed estendere il sistema con facilità.

1.2 Glossario

Al fine di prevenire ambiguità e garantire una comunicazione uniforme e precisa tra i membri del gruppo e i committenti, è stato redatto un Glossario^G apposito. Per facilitare la lettura del presente documento, i termini tecnici o dotati di un significato specifico all'interno del dominio di progetto sono contrassegnati da una lettera «G» posta in apice (es. Parola^G). La definizione estesa di tali termini è reperibile nel documento citato tra i riferimenti informativi.

1.3 Scopo del prodotto

Il sistema CodeGuardian è progettato per condurre analisi approfondite sulla qualità di repository GitHub^G, garantendo un controllo rigoroso sulla gestione degli accessi e sull'autorizzazione all'avvio delle procedure di scansione.

La piattaforma si propone come uno strumento strategico per team di sviluppo multidisciplinari, consentendo il monitoraggio centralizzato dello stato del software e la generazione di metriche aggregate su molteplici progetti analizzati.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- Norme Di Progetto^G V1.1.0
https://byte-holders.github.io/Documentazione/RTB/Norme_Di_Progetto-V1.1.0.pdf
- Capitolato
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C2.pdf>

1.4.2 Riferimenti informativi

- Glossario V1.0.0
<https://byte-holders.github.io/Documentazione/RTB/Glossario-V1.0.0.pdf>

2 Tecnologie

2.1 Linguaggi di programmazione

2.1.1 TypeScript

- **Descrizione:** [Versione:]

Linguaggio di programmazione open-source sviluppato da Microsoft come superset tipato di JavaScript. Introduce la tipizzazione statica opzionale, interfacce e classi, permettendo di individuare errori in fase di compilazione anziché a runtime. La sua adozione facilita lo sviluppo di applicazioni di grandi dimensioni, migliorando il supporto agli IDE e la manutenibilità complessiva.

Documentazione: <https://www.typescriptlang.org/docs/>

2.2 Framework

2.2.1 NestJS

- **Descrizione:** [Versione:]

Framework progressivo per la costruzione di applicazioni lato server efficienti e scalabili su NodeJS. Utilizza TypeScript di default e implementa un'architettura modulare ispirata ad Angular, facilitando l'uso di Dependency Injection e la separazione delle responsabilità.

Documentazione: <https://docs.nestjs.com/>

2.2.2 Langgraph

- **Descrizione:** [Versione:]

Libreria costruita sopra LangChain per la creazione di workflow ciclici e agenti AI basati su grafi di stato, permettendo un controllo granulare sulla logica di esecuzione e sulla memoria dei modelli linguistici (LLM).

Documentazione: <https://langchain-ai.github.io/langgraphjs/>

2.3 Librerie

2.3.1 ReactJS

- **Descrizione:** [Versione:]

Libreria JavaScript per la creazione di interfacce utente basata su componenti. Utilizza un DOM virtuale per ottimizzare le prestazioni e supporta un approccio dichiarativo per la gestione dello stato della UI.

Documentazione: <https://react.dev/>

2.3.2 TanstackRouter

- **Descrizione:** [Versione:]

Sistema di routing tipizzato per React, che garantisce la sicurezza dei tipi (type-safety) per parametri di ricerca e rotte, migliorando l'affidabilità della navigazione interna.

Documentazione: <https://tanstack.com/router/latest>

2.3.3 mongoose

- **Descrizione:** [Versione:]

Libreria ODM (Object Data Modeling) per MongoDB e NodeJS, che fornisce una soluzione basata su schemi per modellare i dati dell'applicazione e gestirne la validazione.

Documentazione: <https://mongoosejs.com/docs/>

2.3.4 axios

- **Descrizione:** [Versione:]

Client HTTP basato su promise per browser e NodeJS, utilizzato per effettuare richieste asincrone verso API esterne o il backend in modo semplice e configurabile.

2.3.5 isomorphic-git

- **Descrizione:** [Versione:]

Implementazione di Git interamente in JavaScript che permette di interagire con i repository sia in ambiente Node.js che direttamente nel browser.

2.3.6 react-markdown

- **Descrizione:** [Versione:]

Componente React utilizzato per il rendering sicuro di stringhe in formato Markdown all'interno dell'interfaccia utente, utile per la visualizzazione di documentazione o messaggi formattati.

2.3.7 dotenv

- **Descrizione:** [Versione:]

Modulo che carica le variabili d'ambiente da un file `.env` in `process.env`, permettendo di gestire configurazioni sensibili separatamente dal codice sorgente.

2.4 Testing

2.4.1 jest

- **Descrizione:** [Versione:]

Framework di test JavaScript focalizzato sulla semplicità, ampiamente utilizzato per unit test e integration test grazie alla sua ampia suite di strumenti inclusi.

2.4.2 vitest

- **Descrizione:** [Versione:]

Framework di testing ultra-veloce basato su Vite, progettato per offrire un'esperienza di sviluppo rapida e nativa per progetti moderni in TypeScript.

2.4.3 playwright

- **Descrizione:** [Versione:]

Strumento per il testing end-to-end che consente l'automazione dei browser moderni, fondamentale per verificare il corretto funzionamento dei flussi utente completi.

Documentazione: <https://playwright.dev/>

2.5 Infrastruttura

2.5.1 Docker

- **Descrizione:** [Versione:]

Piattaforma di containerizzazione che permette di pacchettizzare l'applicazione e le sue dipendenze in un'unità standardizzata per garantire coerenza tra sviluppo e produzione.

Documentazione: <https://docs.docker.com/>

2.5.2 AWS Amplify

- **Descrizione:** [Versione:]

Set di strumenti e servizi cloud per lo sviluppo e il deployment rapido di applicazioni web e mobile full-stack su infrastruttura Amazon Web Services.

2.5.3 AWS ECS

- **Descrizione:** [Versione:]

Amazon Elastic Container Service, un servizio di orchestrazione di container altamente scalabile che permette di gestire l'esecuzione di container Docker nel cloud.

2.5.4 AWS EC2

- **Descrizione:** [Versione:]

Amazon Elastic Compute Cloud, servizio che fornisce capacità di calcolo scalabile sotto forma di istanze di server virtuali per ospitare servizi e applicazioni.

3 Architettura

3.1 Architettura di deployment

3.2 Architettura logica

3.2.1 Frontend

3.2.2 Backend

3.2.3 Container